

Student 27

R Yazhini

192521190RD

4 / 4 submitted

Q1. Intelligent Customer Feedback Analysis

CODE

```
def preprocess_text(text):
    """Normalizes case, strips whitespace, and removes punctuation."""
    cleaned = text.lower().strip()
    for char in [".", "!", "?", ";", ":", ", ", "'", ""]:
        cleaned = cleaned.replace(char, "")
    return cleaned

def analyze_sentiment(feedback, positive_words, negative_words):
    """Counts positive and negative keywords and determines sentiment."""
    cleaned = preprocess_text(feedback)
    words = cleaned.split()
    pos_count = 0
    neg_count = 0
    for word in words:
        if word in positive_words:
            pos_count += 1
        elif word in negative_words:
            neg_count += 1
    if pos_count > neg_count:
        sentiment = "Positive"
    elif neg_count > pos_count:
        sentiment = "Negative"
    else:
        sentiment = "Neutral"
    return sentiment, pos_count, neg_count

def process_feedback_entries():
    """Handles input/output loop for multiple feedback entries."""
    positive_words = ["good", "great", "excellent", "love", "best"]
    negative_words = ["bad", "poor", "terrible", "worst", "hate", "slow"]
    try:
        n = int(input())
        for _ in range(n):
            entry = input().strip()
            sentiment, pos_c, neg_c = analyze_sentiment(
                entry, positive_words, negative_words
            )
            print(f"Result: {sentiment} (Positive: {pos_c}, Negative: {neg_c})")
    except EOFError:
        pass
process_feedback_entries()
```

INPUT

2
The product quality is goof and the service was great!
The delivery was very slow and bad.

OUTPUT

Result: Positive (Positive: 1, Negative: 0)
Result: Negative (Positive: 0, Negative: 2)

Q2. Comprehensive Password Strength Checker

CODE

```
def check_password(password):
    if not password:
        return 0, 0, 0, 0, "Weak"
    upper = 0
    lower = 0
    digit = 0
    special = 0
    symbols = "!@#$%^&*()-_+=+[]{};,:./"
    for ch in password:
        if ch.isupper():
            upper += 1
        elif ch.islower():
            lower += 1
        elif ch.isdigit():
            digit += 1
        elif ch in symbols:
            special += 1
    score = 0
    if len(password) >= 8:
        score += 1
    if upper > 0:
        score += 1
    if lower > 0:
        score += 1
    if digit > 0:
        score += 1
    if special > 0:
        score += 1
    if score <= 2:
        strength = "Weak"
    elif score == 3:
        strength = "Moderate"
    elif score == 4:
        strength = "Strong"
    else:
        strength = "Very Strong"
    return upper, lower, digit, special, strength
def feedback(password, upper, lower, digit, special):
    print("\nSuggestions:")
    if len(password) < 8:
        print("- Password should contain at least 8 characters.")
    if upper == 0:
        print("- Add at least one uppercase letter.")
    if lower == 0:
        print("- Add at least one lowercase letter.")
    if digit == 0:
        print("- Add at least one digit.")
    if special == 0:
        print("- Add at least one special symbol.")
password = input()
upper, lower, digit, special, strength = check_password(password)
print("Password Analysis")
print("Uppercase Letters:", upper)
print("Lowercase Letters:", lower)
print("Digits:", digit)
print("Special Symbols:", special)
print("Password Strength:", strength)
feedback(password, upper, lower, digit, special)
```

Q3. Structured Server Log Error Analyser

CODE

```
def extract_errors(logs):
    """Filters log entries to find error lines using string searching methods."""
    errors = []
    for log in logs:
        if log.startswith("ERROR") or log.find("ERROR") != -1 or "error" in log.lower():
            errors.append(log)
    return errors

def remove_duplicates(errors):
    """Removes duplicate error messages while preserving order."""
    unique = []
    for error in errors:
        if error not in unique:
            unique.append(error)
    return unique

def count_errors(errors):
    """Counts and categorizes errors by type."""
    file_err = 0
    network_err = 0
    db_err = 0
    other_err = 0

    for error in errors:
        e = error.lower()
        if "file" in e:
            file_err += 1
        elif "network" in e:
            network_err += 1
        elif "database" in e or "db" in e:
            db_err += 1
        else:
            other_err += 1

    return file_err, network_err, db_err, other_err

try:
    n = int(input())
    logs = [input().strip() for _ in range(n)]
    error_logs = extract_errors(logs)
    unique_errors = remove_duplicates(error_logs)
    print("Unique Error logs:")
    for err in unique_errors:
        print(err)
    f, n_err, d, o = count_errors(unique_errors)
    print("\nError Summary")
    print("File Errors:", f)
    print("Network Errors:", n_err)
    print("Database Errors:", d)
    print("Other Errors:", o)

except EOFError:
    pass
```

INPUT

5
INFO: System booted successfully
ERROR: File not found in directory
ERROR: Network connection timed out
ERROR: File not found in directory
ERROR: Database connection failed

OUTPUT

Q4. Email Domain Frequency Analyser

CODE

```
def extract_domain(email):
    """
    Extracts the domain name from an email address using split('@').
    Handles malformed emails gracefully using conditionals.
    """
    if "@" in email and email.count("@") == 1:
        parts = email.split("@")
        if len(parts) == 2 and parts[0] and parts[1]:
            return parts[1]
    return None

def analyze_domains():
    """
    Reads input emails, counts domain frequencies using loops and conditionals,
    and identifies the maximum frequency domain.
    """
    try:
        n = int(input())
        domains = {}

        for _ in range(n):
            email = input().strip()
            domain = extract_domain(email)

            if domain:
                if domain in domains:
                    domains[domain] += 1
                else:
                    domains[domain] = 1
            else:
                print("Invalid Email")

        print("Domain Frequency:")
        max_domain = ""
        max_count = 0

        for d, count in domains.items():
            print(f"{d} : {count}")
            if count > max_count:
                max_count = count
                max_domain = d

        if max_domain:
            print(f"Most Used Domain: {max_domain}")

    except EOFError:
        pass

# Run the analysis
analyze_domains()
```

INPUT

4
user1@gmail.com
user2@yahoo.com
user3@gmail.com
invalid-email-format

OUTPUT